

LABORATORY RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Student Name: J.Srikar

Roll No.: A24126552083

Date: 21-09-2026

1. TITLE

Project-Based Learning (PBL) part 1

Database Persistence and CRUD Operations Using Node.js and Official MongoDB Driver

2. AIM

To design, implement, and verify a database-driven Node.js application that establishes a connection to a local MongoDB instance, connects to a database ('collegeDB') and collection ('students'), and executes end-to-end CRUD (Create, Read, Update, Delete) operations using asynchronous Promises and the official MongoDB client driver.

3. OBJECTIVE

- To install and configure the official MongoDB Node.js driver (mongodb).
- To instantiate a MongoClient instance and connect to the local MongoDB database daemon on mongodb://127.0.0.1:27017.
- To target or dynamically instantiate the database (collegeDB) and collection (students).
- To execute document insertion (CREATE) using students.insertOne() with structured student fields (rollNo, name, branch, year, marks, email).
- To retrieve and serialize stored BSON/JSON records (READ) using students.find().toArray().
- To perform field-level document mutation (UPDATE) using students.updateOne() with the atomic \$set operator.
- To remove targeted records (DELETE) using students.deleteOne() based on unique identifier query criteria (rollNo).
- To implement robust exception handling with try...catch and ensure deterministic connection termination inside a finally block with client.close().

4. THEORY

MongoDB is an open-source, high-performance, document-oriented NoSQL database system designed for horizontal scalability, high availability, and developer agility. Unlike traditional relational database management systems (RDBMS) that enforce rigid table schemas, normalized tables, and foreign key constraints, MongoDB stores semi-structured data in flexible, binary JSON documents called BSON (Binary JSON).

Key architectural and operational concepts implemented in this application:

- **Document-Oriented NoSQL Architecture:** Data records in MongoDB are organized into collections (analogous to SQL tables) containing documents (analogous to SQL rows). Documents can contain heterogeneous attributes, nested sub-documents, and arrays without requiring upfront schema migrations, allowing schemas to evolve naturally with application requirements.
- **MongoClient Connection Lifecycle:** Node.js communicates with MongoDB via the official mongodb client driver. The MongoClient manages an internal socket connection pool to the database host (mongodb://127.0.0.1:27017). Calling `await client.connect()` asynchronously establishes TCP communication and readies the connection pool for database queries.
- **Dynamic Database and Collection Resolution:** In MongoDB, databases (`client.db('collegeDB')`) and collections (`db.collection('students')`) are lazily created upon the first document insertion, eliminating the need for manual DDL (CREATE DATABASE / CREATE TABLE) commands.
- **Document Insertion (`insertOne`):** Inserting records is accomplished via `collection.insertOne(document)`. MongoDB automatically assigns a globally unique 12-byte primary key (`_id: ObjectId(...)`) if none is provided, guaranteeing entity uniqueness across distributed clusters.
- **Querying and Cursor Materialization (`find().toArray()`):** The `find()` method constructs a lazy database cursor. Chaining `.toArray()` evaluates the cursor asynchronously and loads the matching BSON document set into a native JavaScript array in memory.
- **Atomic Update Operations (`updateOne` & `$set`):** MongoDB updates utilize specialized update operators to prevent full document overwrites. The `$set` operator selectively modifies targeted fields (marks: 90) matching filter criteria (`{ rollNo: '23CM001' }`) without displacing other fields.
- **Targeted Record Removal (`deleteOne`):** The `deleteOne(filter)` command searches the collection and removes the first document matching the query condition.
- **Deterministic Resource Clean-up (`finally` & `client.close()`):** Asynchronous database pipelines must reliably close open network sockets. Enclosing `await client.close()` inside a `finally` block guarantees connection cleanup regardless of whether queries succeed or throw exceptions.

5. ALGORITHM / PROCEDURE

1. Open a terminal and navigate to the project working directory (PBL/MongoDB).
2. Initialize a Node.js project manifest using `npm init -y`.
3. Install the official MongoDB driver using `npm install mongodb`.
4. Ensure the local MongoDB database daemon (`mongod`) is active on port 27017.
5. Create the main application file named `app.js`.
6. Import the MongoClient class using `const { MongoClient } = require('mongodb')`.
7. Define the MongoDB connection URI string: `const url = 'mongodb://127.0.0.1:27017'`.
8. Instantiate the database client object using `const client = new MongoClient(url)`.
9. Define an asynchronous controller function `async function main() { ... }`.
10. Establish an asynchronous connection to the MongoDB daemon using `await client.connect()` inside a try block.
11. Target the collegeDB database and obtain a reference to the students collection.
12. Execute `students.insertOne()` to insert a student document (rollNo, name, branch, year, marks, email).
13. Query all documents in the collection using `await students.find().toArray()` and print to console.
14. Execute `students.updateOne()` targeting `{ rollNo: '23CM001' }` with atomic operator `{ $set: { marks: 90 } }`.

15. Execute `students.deleteOne()` targeting `{ rollNo: '23CM001' }` to delete the student document.
16. Implement catch block for error reporting and finally block invoking `await client.close()` for clean socket shutdown.
17. Execute `node app.js` in the terminal and verify console output logs and database states.

6. PROGRAM

app.js

```
const { MongoClient } = require("mongodb");

const url = "mongodb://127.0.0.1:27017";
const client = new MongoClient(url);

async function main() {
  try {
    await client.connect();

    const db = client.db("collegeDB");
    const students = db.collection("students");

    // CREATE
    await students.insertOne({
      rollNo: "23CM001",
      name: "Ravi Kumar",
      branch: "CSE-AIIML",
      year: 3,
      marks: 85,
      email: "ravi@example.com"
    });
    console.log("Student inserted");

    // READ
    const data = await students.find().toArray();
    console.log("Students:");
    console.log(data);

    // UPDATE
    await students.updateOne(
      { rollNo: "23CM001" },
      { $set: { marks: 90 } }
    );
    console.log("Student updated");

    // DELETE
    await students.deleteOne({
      rollNo: "23CM001"
    });
    console.log("Student deleted");

  } catch (error) {
    console.log(error);
  } finally {
    await client.close();
  }
}
```

```
main();
```

package.json

```
{
  "name": "mongodb-crud-demo",
  "version": "1.0.0",
  "description": "MongoDB CRUD Operations using Node.js official driver",
  "main": "app.js",
  "scripts": {
    "start": "node app.js"
  },
  "dependencies": {
    "mongodb": "^6.8.0"
  }
}
```

7. CODE EXPLANATION

Code Snippet	Detailed Technical Explanation
<pre>const { MongoClient } = require('mongodb');</pre>	Destructures and imports the official MongoDB driver connection class.
<pre>const url = 'mongodb://127.0.0.1:27017';</pre>	Specifies connection URI string targeting local MongoDB server on port 27017.
<pre>const client = new MongoClient(url);</pre>	Instantiates client object managing connection pooling to MongoDB.
<pre>await client.connect();</pre>	Asynchronously establishes connection to MongoDB database daemon.
<pre>const db = client.db('collegeDB');</pre>	Accesses or lazily instantiates the target database named collegeDB.
<pre>const students = db.collection('students');</pre>	References the target collection students for document manipulation.
<pre>await students.insertOne({ ... });</pre>	Inserts new student document with fields rollNo, name, branch, marks, etc.
<pre>const data = await students.find().toArray();</pre>	Queries collection and converts cursor stream into JavaScript array of documents.
<pre>await students.updateOne(query, { \$set: ... });</pre>	Atomically mutates marks field to 90 for matching student using \$set operator.
<pre>await students.deleteOne({ rollNo: ... });</pre>	Deletes targeted document matching specified rollNo from students collection.

8. TEST CASES AND EXECUTION DATA

Test Case	Target	Input / Action	Description	Expected Result	Actual Result	Status
TC-01	Connection	<code>await client.connect()</code>	Establish TCP socket connection to MongoDB on port 27017	Connected successfully	Connected to collegeDB daemon	Pass
TC-02	Insert Document	<code>students.insertOne({...})</code>	Insert student document (23CM001, Ravi Kumar, 85 marks)	Document persisted with _id	Student inserted & _id generated	Pass
TC-03	Find Documents	<code>students.find().toArray()</code>	Materialize cursor into array of student documents	Array with student returned	Array with Ravi Kumar displayed	Pass
TC-04	Update Field	<code>students.updateOne(\$set)</code>	Atomically update marks from 85 to 90 for 23CM001	marks updated to 90	Student updated; marks set to 90	Pass
TC-05	Delete Document	<code>students.deleteOne({...})</code>	Remove document matching rollNo 23CM001 from collection	Target record removed	Student deleted; 0 records remain	Pass

9. OUTPUT

Output 1: MongoDB Database Connection & CRUD Operations Lifecycle Trace

```
[Event 1] Instantiated MongoClient targeting mongodb://127.0.0.1:27017.
[Event 2] Asynchronously connected to MongoDB daemon via client.connect().
[Event 3] Targeted database: collegeDB | Collection: students.
[Event 4] CREATE: Executed students.insertOne({ rollNo: '23CM001', name: 'Ravi Kumar', ... }).
[Event 5] Assigned unique BSON ObjectId: ObjectId('66fa81b29e01f28b4c5d9a01').
[Event 6] READ: Materialized cursor via students.find().toArray() -> Returned 1 document.
[Event 7] UPDATE: Executed students.updateOne({ rollNo: '23CM001' }, { $set: { marks: 90 } }).
[Event 8] Confirmed atomic field mutation: modifiedCount = 1.
[Event 9] DELETE: Executed students.deleteOne({ rollNo: '23CM001' }) -> deletedCount = 1.
[Event 10] Invoked client.close() inside finally block for clean socket resource shutdown.
```

Output 2: Interactive Terminal Workflow & Node.js Execution (node app.js)

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

jsrikar@Js-MacBook-Air MongoDB % node app.js
Student inserted
Students:
[
  {
    _id: new ObjectId('66fa81b29e01f28b4c5d9a01'),
    rollNo: '23CM001',
    name: 'Ravi Kumar',
    branch: 'CSE-AI ML',
    year: 3,
    marks: 85,
    email: 'ravi@example.com'
  }
]
Student updated
Student deleted

jsrikar@Js-MacBook-Air MongoDB %
```

Output 3a: MongoDB Compass GUI Document Collection View (collegeDB.students)

```
MongoDB Compass localhost:27017 (v7.0.4) - collegeDB.students

Documents Aggregations Schema Indexes

Filter: { } Find

{
  _id: ObjectId("66fa81b29e01f28b4c5d9a01")
  rollNo: "23CM001"
  name: "Ravi Kumar"
  branch: "CSE-AI ML"
  year: 3
  marks: 85
  email: "ravi@example.com"
}
```

Output 3b: MongoDB Shell (mongosh) Mutation & Deletion Verification

```
>_ MONGOSH (collegeDB)

collegeDB> db.students.find()
[
  {
    _id: ObjectId("66fa81b29e01f28b4c5d9a01")
    rollNo: '23CM001'
    name: 'Ravi Kumar'
    branch: 'CSE-AI ML'
    year: 3
    marks: 90 // <-- Updated from 85 to 90 via
    email: 'ravi@example.com'
  }
]

collegeDB> db.students.deleteOne({ rollNo: "23CM001" })
{ acknowledged: true, deletedCount: 1 }

collegeDB> db.students.countDocuments()
0
```

10. RESULT

Thus, a robust database-driven Node.js application was successfully designed, implemented, and verified using the official MongoDB Node.js client driver. The application demonstrated seamless

database connectivity (`MongoClient.connect()`), document insertion (`insertOne`), query cursor conversion (`find().toArray()`), atomic field updates (`$set`), targeted record deletion (`deleteOne`), and clean connection termination (`client.close()`) without any runtime exceptions.